INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 7: Typical Convolutional Neural Networks

Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

Tsinghua University

Last lecture review

I. Optimizers

Adagrad

$$\boldsymbol{c} = \boldsymbol{c} + \boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

RMSProp

$$\boldsymbol{c} = \gamma \boldsymbol{c} + (1 - \gamma)\boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{g}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

Adam

$$\boldsymbol{m} = \beta_1 \boldsymbol{m} + (1 - \beta_1)\boldsymbol{g}$$

$$\boldsymbol{c} = \beta_2 \boldsymbol{c} + (1 - \beta_2)\boldsymbol{g}^2$$

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{c} + \epsilon}}$$

II. Techniques for dealing with overfitting

Early stop | Dropout | Data augmentation

III. Batch normalization

$$\hat{x}_i = \frac{x_i - \mathrm{E}[x_i]}{\sqrt{\mathrm{Var}[x_i]}} \qquad y_i = \gamma_i \hat{x}_i + \beta_i$$

Handout

Choosing Hyperparameters

Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

Tsinghua University

Paper presentation

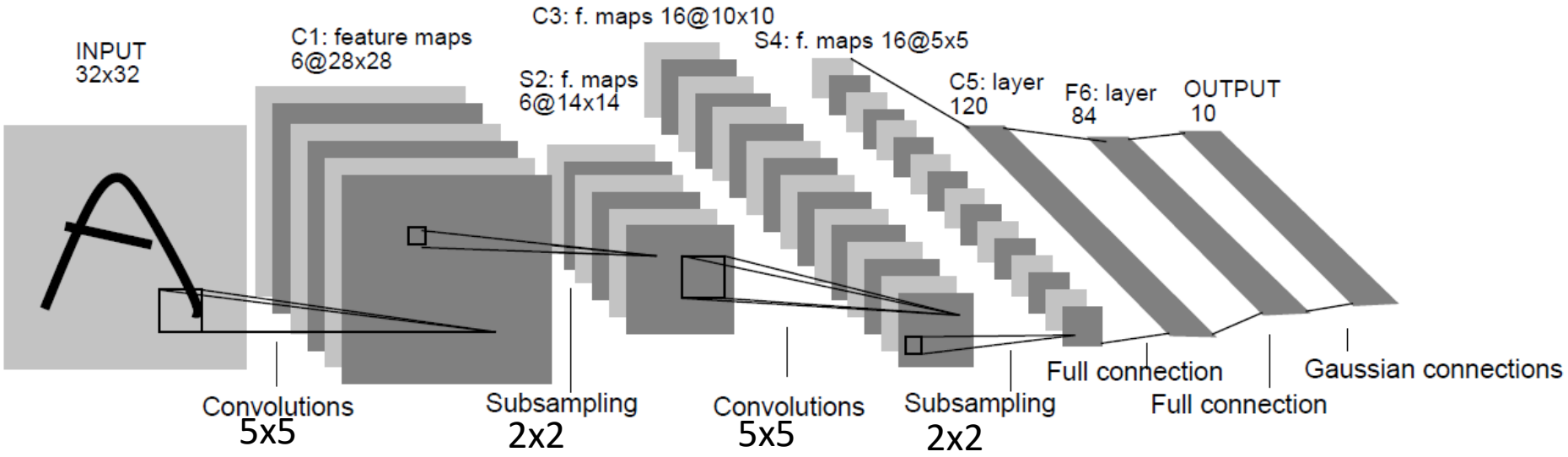
- Form groups of 2-3 and every group prepares a 5-minute presentation with slides for one of the following papers
  - Santurkar et al. “How Does Batch Normalization Help Optimization? ” NeurIPS 2018
  - Li, Chen, Hu, Yang, “Understanding the Disharmony Between Dropout and Batch Normalization by Variance Shift,” CVPR 2019

Outline

I. Popular models
II. Lightweight models
III. Model compression and quantization*
IV. Summary

LeNet-5

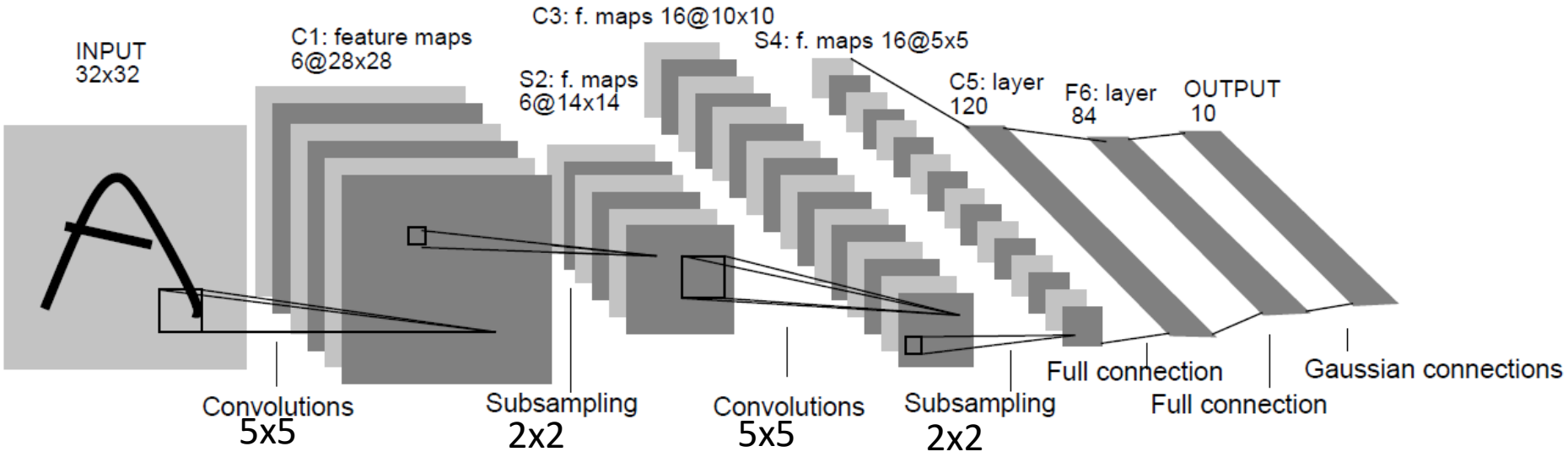
LeCun, Bottou, Bengio, Haffner, 1998



- Local connections and weight sharing
- C layers: convolution
  - Output $y_i = f(\sum_\Omega w_j x_j + b)$ where Ω is the patch size, $f(\cdot)$ is the sigmoid function, w and b are parameters
- S layers: subsampling (avg pooling)
  - Output $y_i = f(w \sum_\Omega x_j + b)$ where Ω is the pooling size

LeNet-5

LeCun, Bottou, Bengio, Haffner, 1998



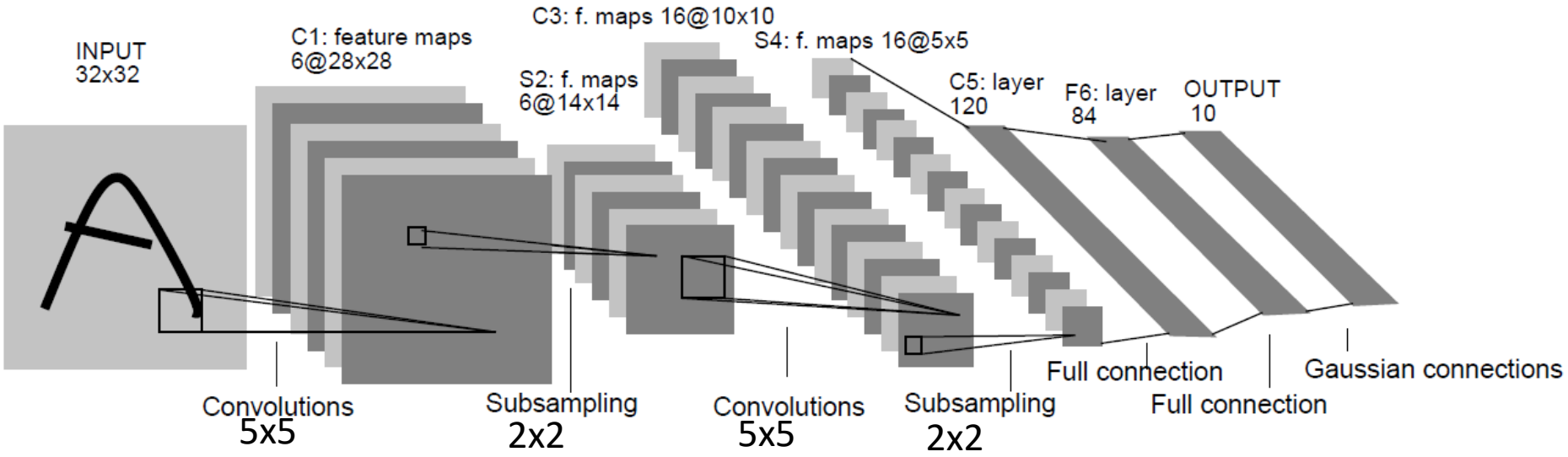
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

TABLE I

EACH COLUMN INDICATES WHICH FEATURE MAP IN S2 ARE COMBINED BY THE UNITS IN A PARTICULAR FEATURE MAP OF C3.

LeNet-5

LeCun, Bottou, Bengio, Haffner, 1998



- Output layer: RBF units
- Loss function:
  - Maximum likelihood with modifications
- Train with back-prop



RBF centers

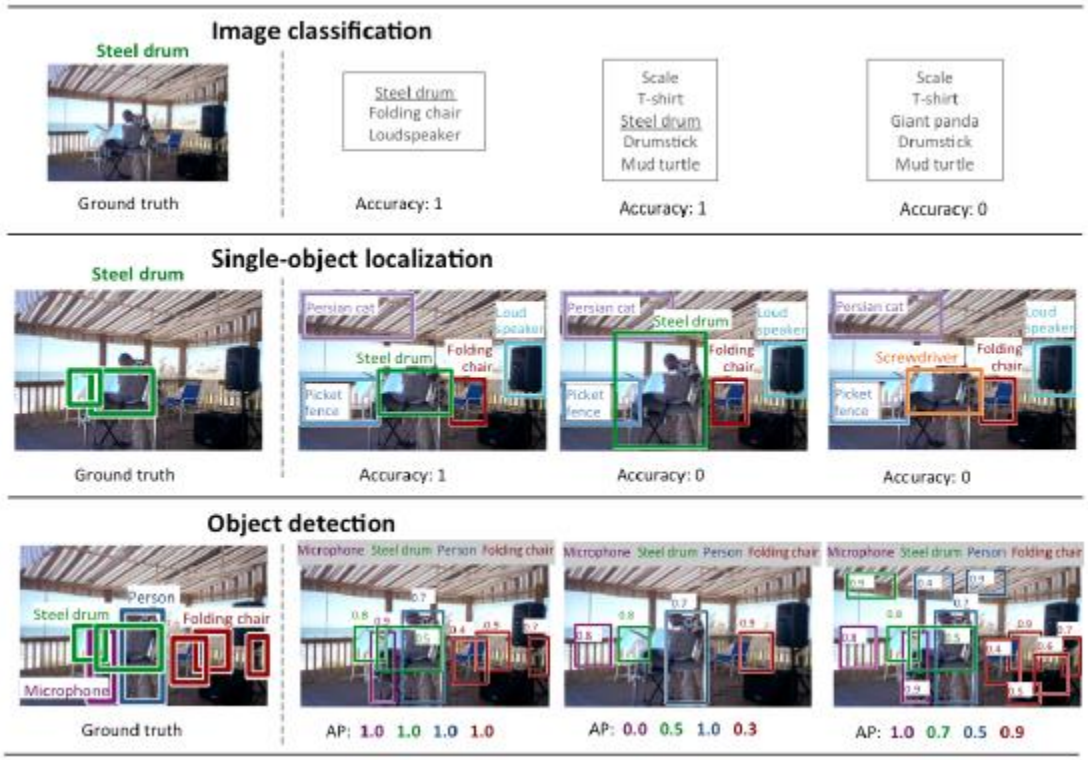
ImageNet comptition (ILSVRC)

Tasks

2010-

2011-

2013-



Top-1
Top-5 (preferred)

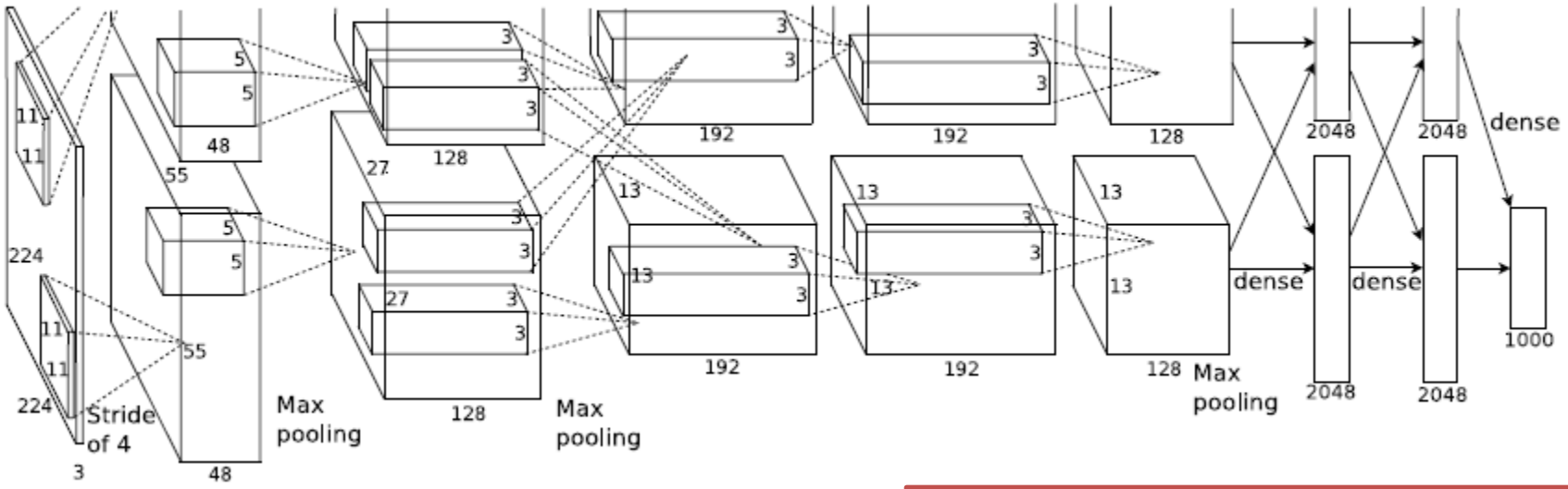
Two human expert: 5.1%, 12%

The first column shows the ground truth labeling on an example image, and the next three show three sample outputs with the corresponding evaluation score.

Russakovsky, et al., 2014

AlexNet

Krizhevsky, Sutskever and Hinton, NIPS, 2012



- Classification: 1000 classes, 1.2 million training images
- In total: 60 million parameters

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

Since then, CNN dominates computer vision society

- In 2013, the vast majority of teams used CNN.
- In 2014 & 2015, almost all teams used convolutional neural networks.

VGG net

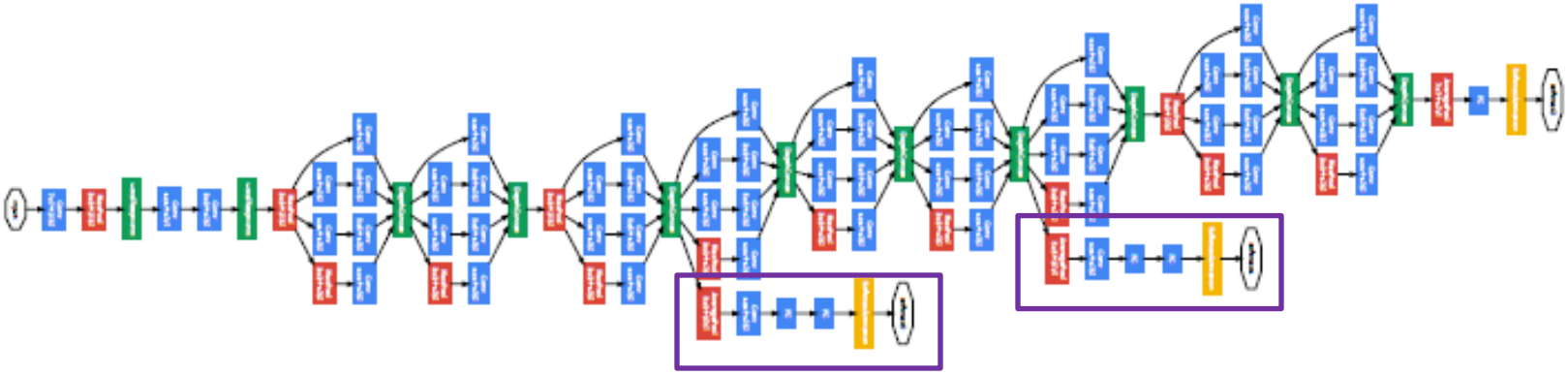
Simonyan, Zisserman, 2015

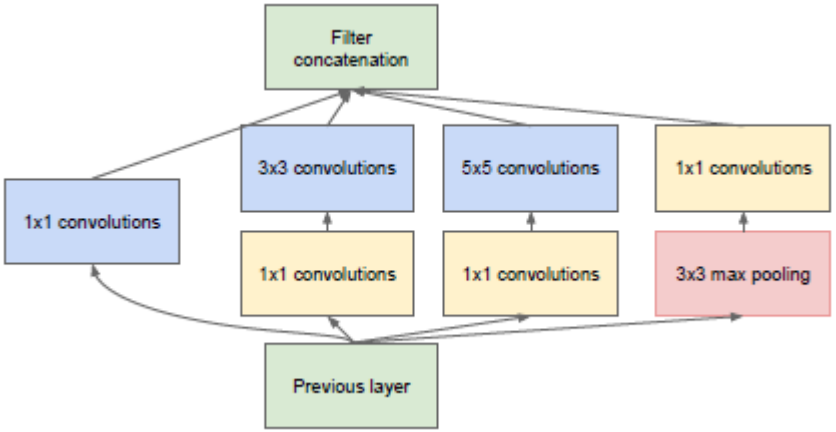
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

- 3*3 filters are extensively used
- GPU implement.

GoogLeNet (Inception-v1)

Szegedy, et al., 2014





- Multiple sizes in the same layer
- 1×1 conv are used to reduce the number of channels

- 22 weight layers
- Small filters (1x1, 3x3, 5x5)
- Two auxiliary classifiers for BP algorithm
- A cpu-based implementation on distributed system

Extensions

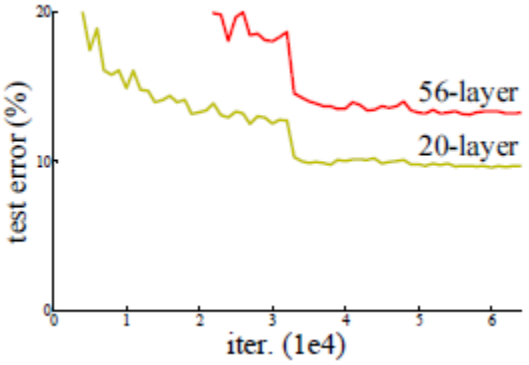
- There are extensions to this model
  - Inception-v2
  - Inception-v3

Szegedy, Vanhoucke, Ioffe et al., Rethinking the Inception Architecture for Computer Vision, CVPR, 2016

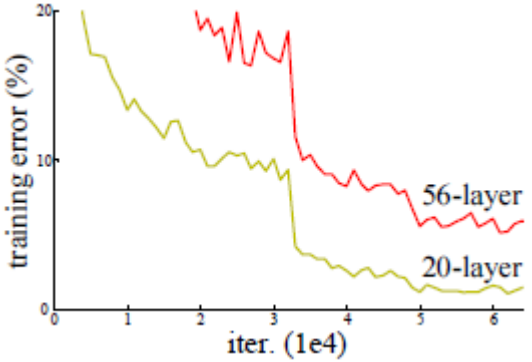
More layers, better results?

	Weight layers	Top-5 error rate
AlexNet	8	17.0%
VggNet	19	7.5%
GoogLeNet	22	6.67%

More layers, better results?



Is it caused by over-fitting?

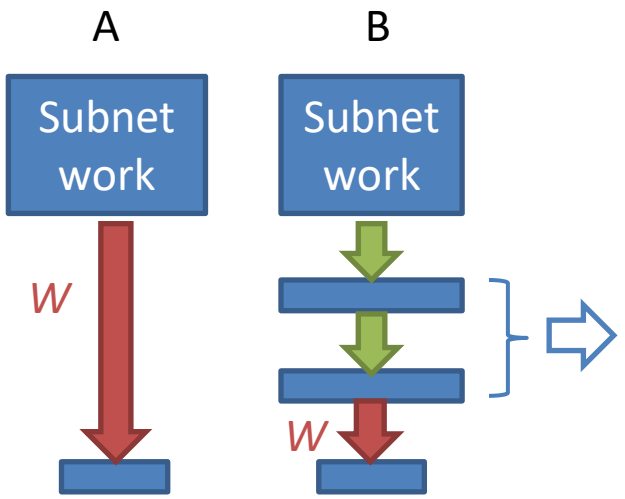


He et al., 2016

Deep residual network (ResNet)

He et al., 2016

Consider two models



Error of B should not be larger than that of A!

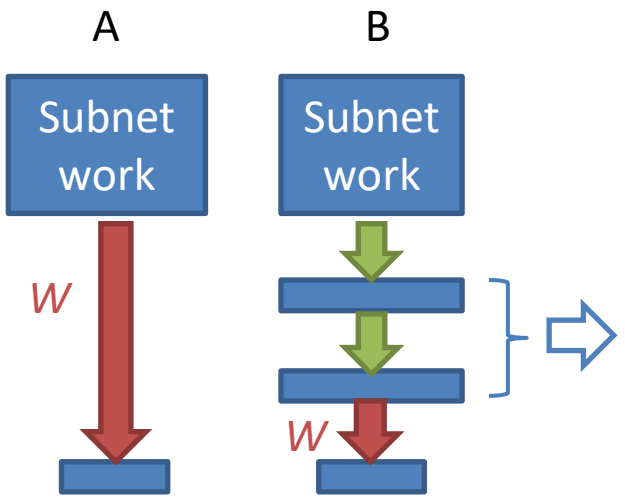
- If they are identity mappings, then A and B are equivalent
- If they include identity mapping as special cases, the capacity of B is larger than A

The assumptions may not hold

Deep residual network (ResNet)

He et al., 2016

Consider two models



Error of B should not be larger than that of A!

- If they are identity mappings, then A and B are equivalent
- If they include identity mapping as special cases, the capacity of B is larger than A

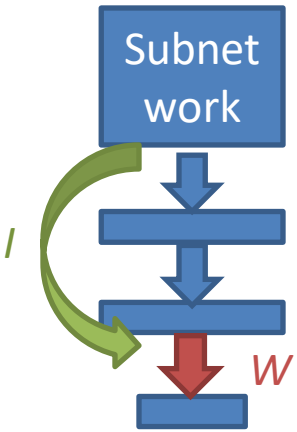
The assumptions may not hold

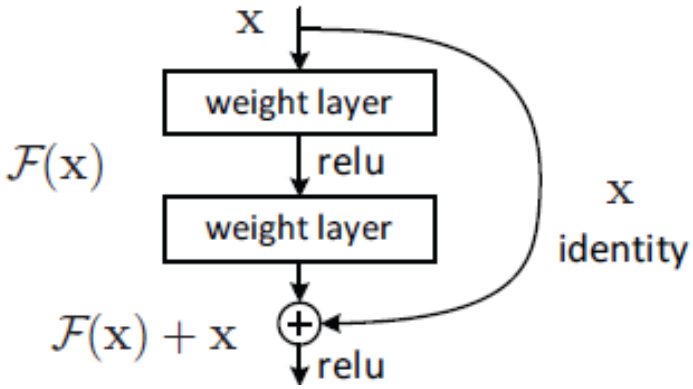
It might be difficult for nonlinear layers to approx. the identity mapping

Deep residual network (ResNet)

He et al., 2016

- If this is the case, let's explicitly use the identity mapping





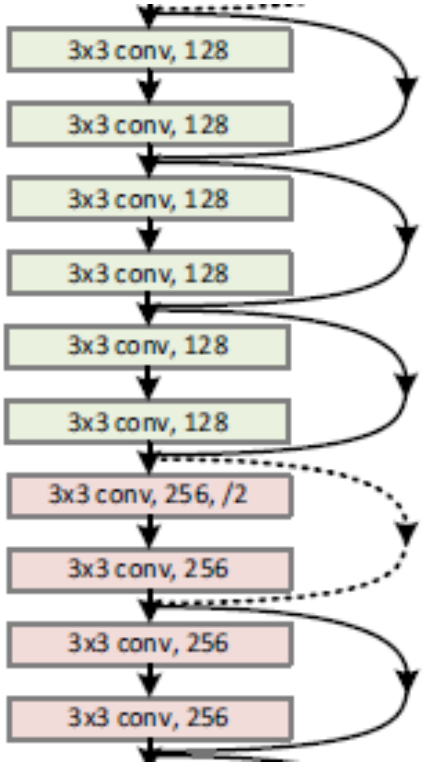
- The nonlinear mapping from input to output $H(x)$ has two parts

$$H(x) = F(x) + x$$

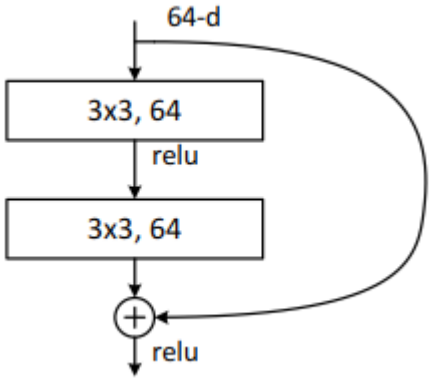
- Then the (two) weight layers are learning $F(x)$, that is the residual $H(x) - x$

Deep residual network (ResNet)

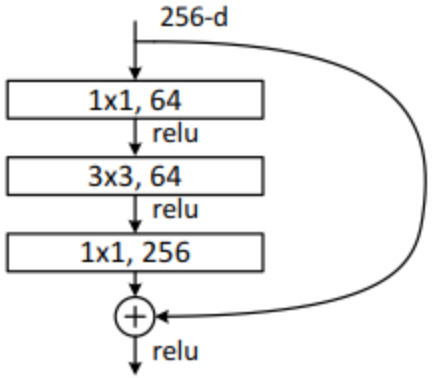
He et al., 2016

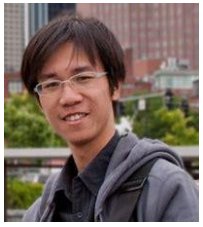


For ResNet34



For ResNet50/101/152





Kaiming He

Best CVPR 2016 paper

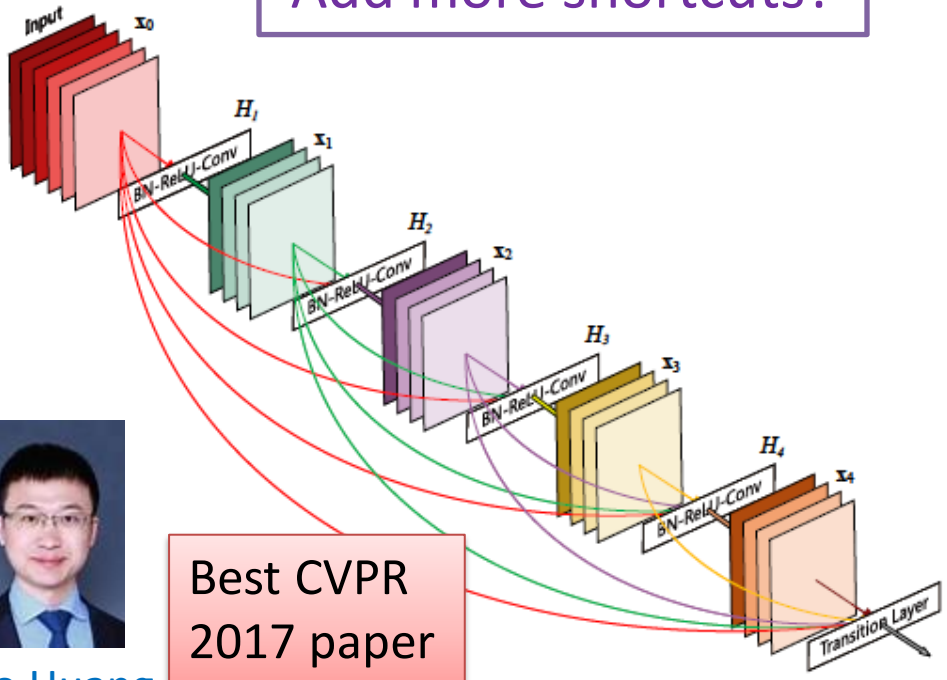
method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	**3.57**

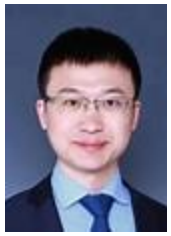
152-layer

DenseNet

Huang et al., CVPR 2017

Add more shortcuts?

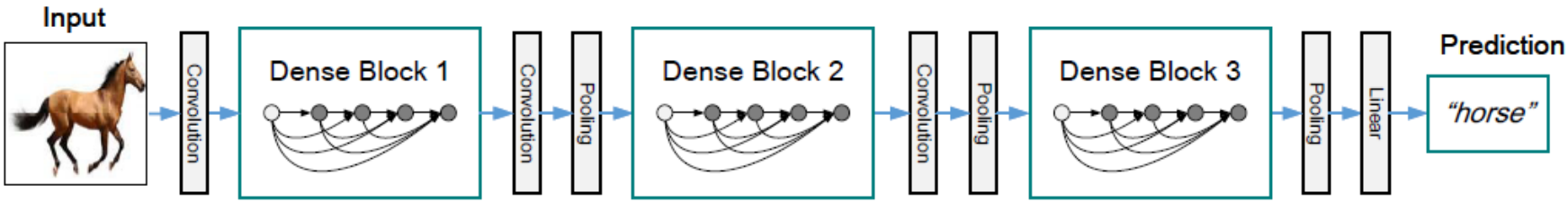




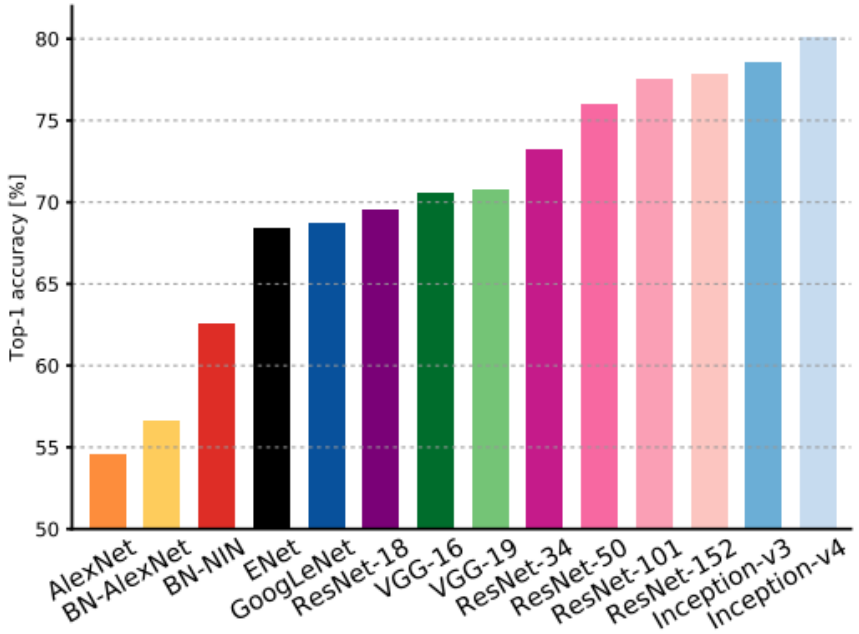
Gao Huang

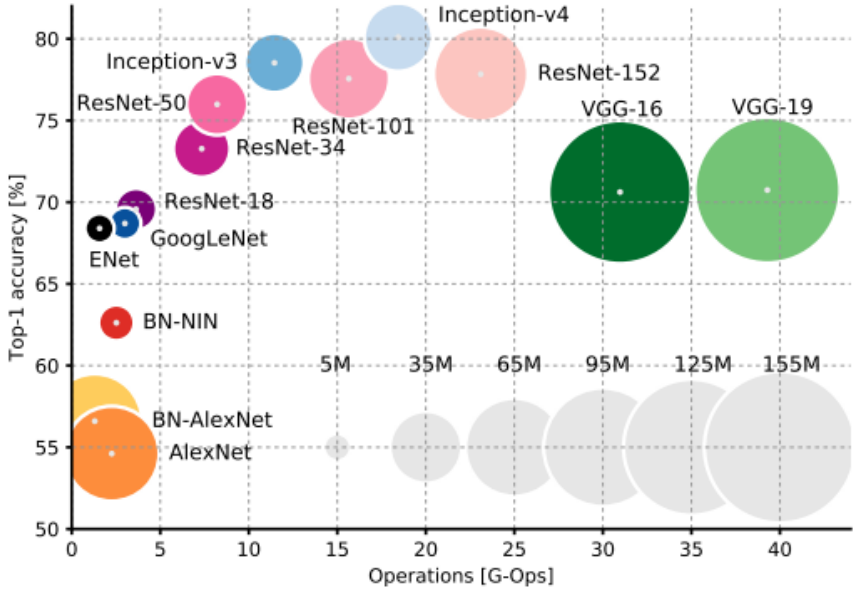
Best CVPR 2017 paper

- Each layer takes all preceding feature-maps as input, which are *concatenated* together
- An L-layer net has $\frac{L(L+1)}{2}$ connections
- Each layer outputs k feature maps and k is small (e.g., 12)



Results on ImageNet challenge





Canziani, Paszke, Culurciello, arXiv:1605.07678v4, 2017

Summary of part I

AlexNet

⇩ Deeper

VGGNet

⇩ Deeper

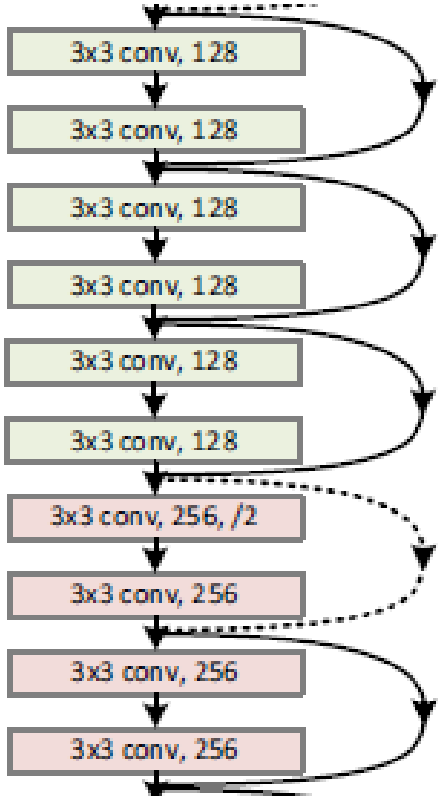
Inception v1-v3

⇩ Add shortcuts

ResNet

⇩ More shortcuts

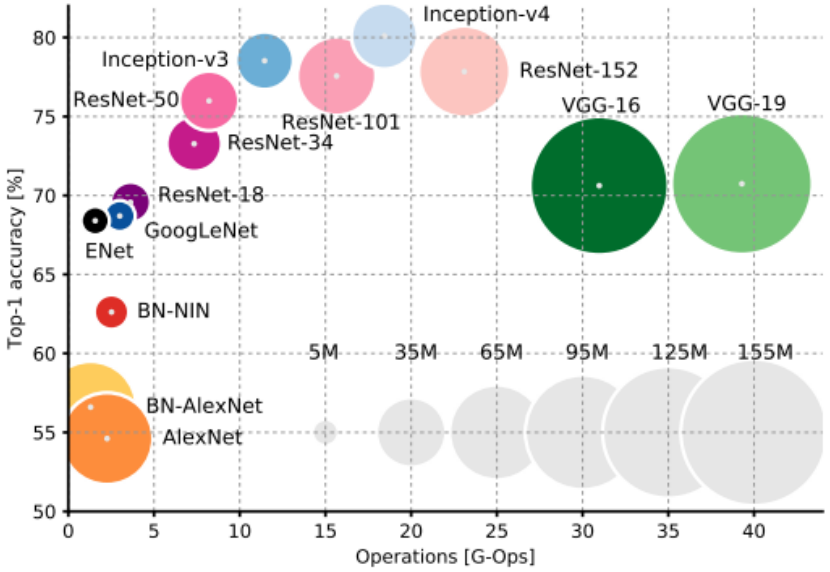
DenseNet

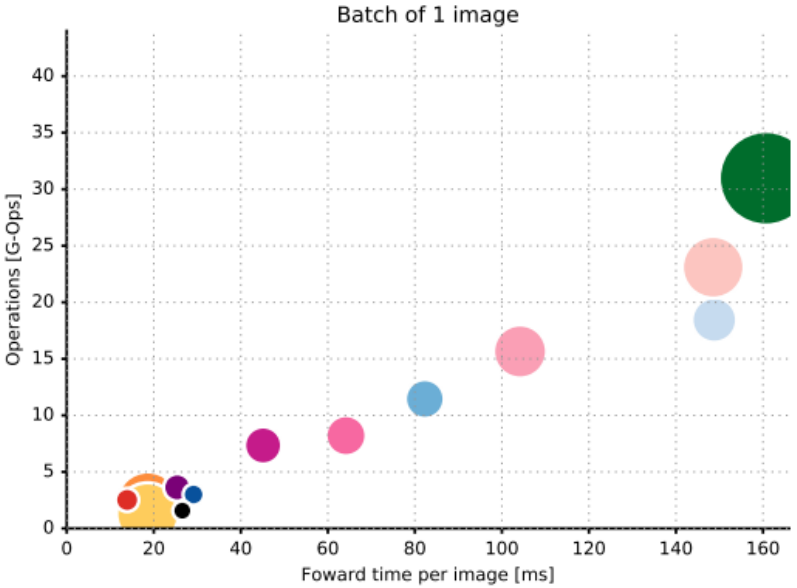


Outline

I. Popular models
II. Lightweight models
III. Model compression and quantization*
IV. Summary

Problems with the models discussed so far





1 T-Flop/s 256-core NVIDIA Maxwell GPU

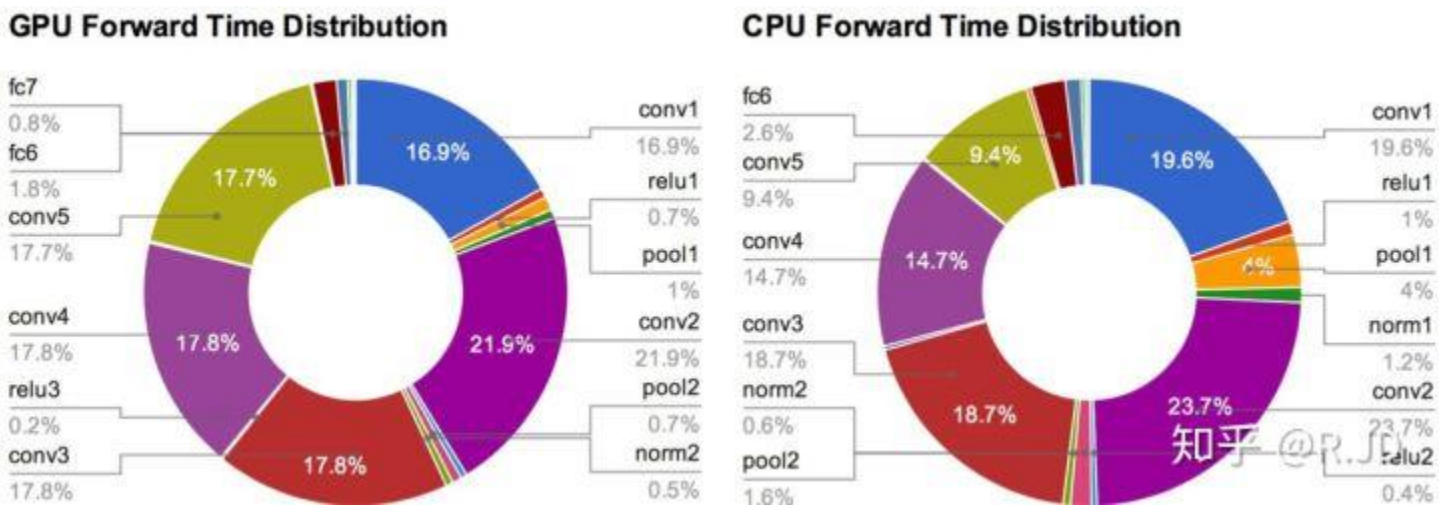
Canziani, Paszke, Culurciello, arXiv:1605.07678v4, 2017

The models are very large

Inference is slow

Reduce the number of parameters and computational cost

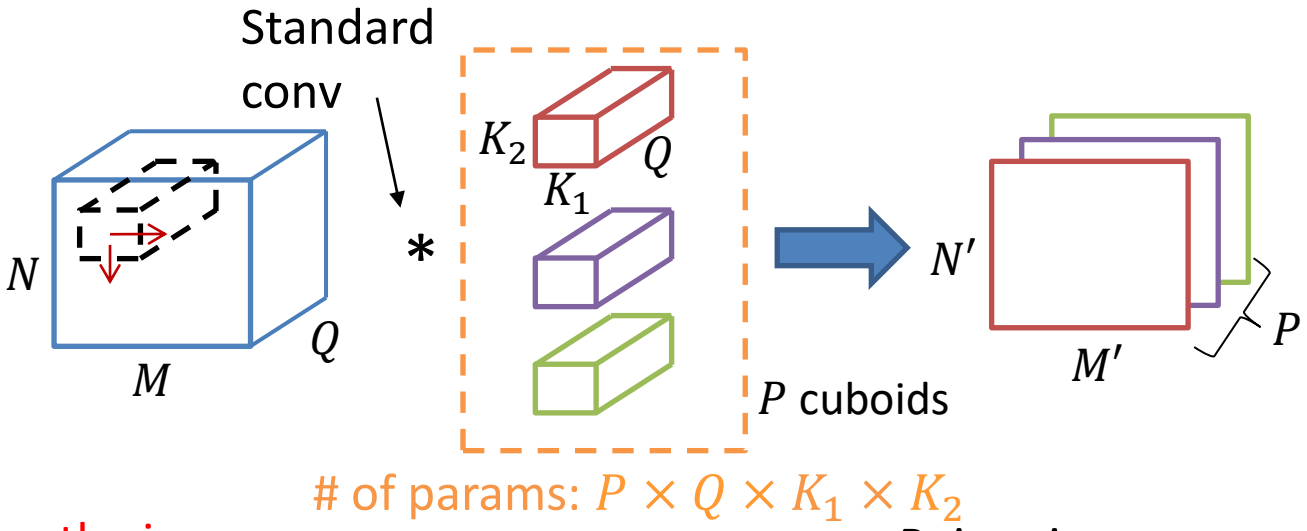
Reduce the computational cost



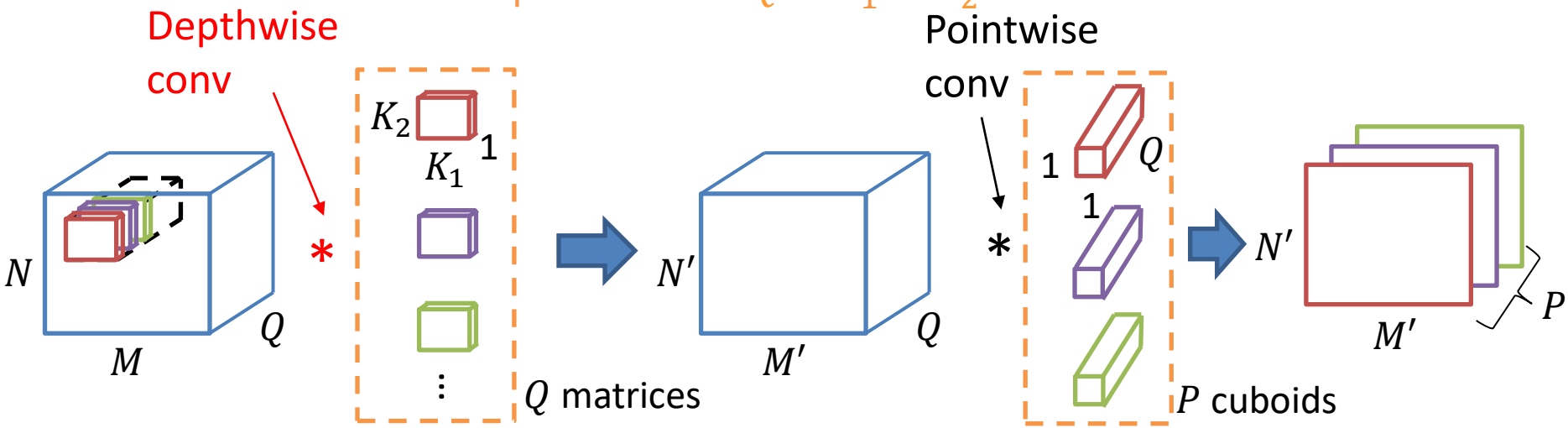
Most computations are spent on conv layers

Reduce the computational cost in conv layers

Depthwise separable convolution



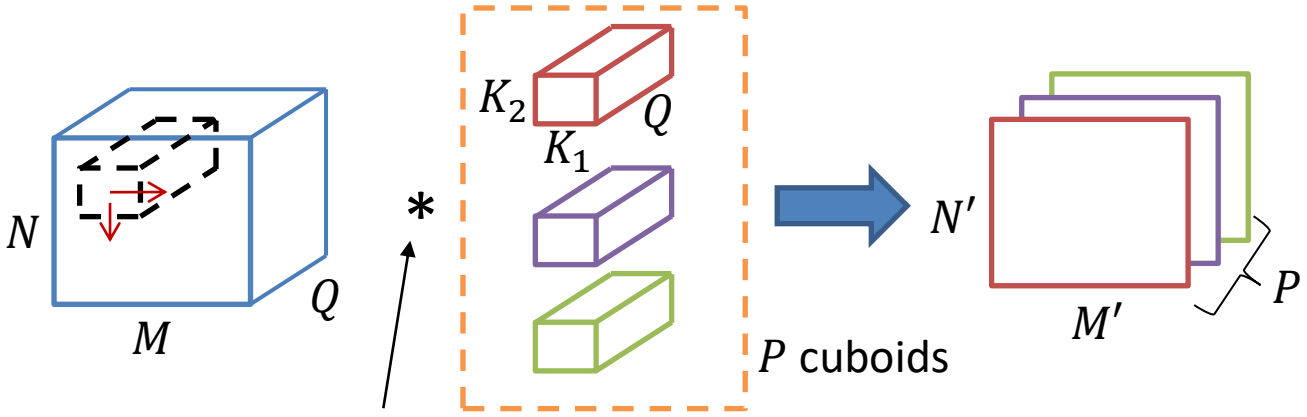
of params: $P \times Q \times K_1 \times K_2$



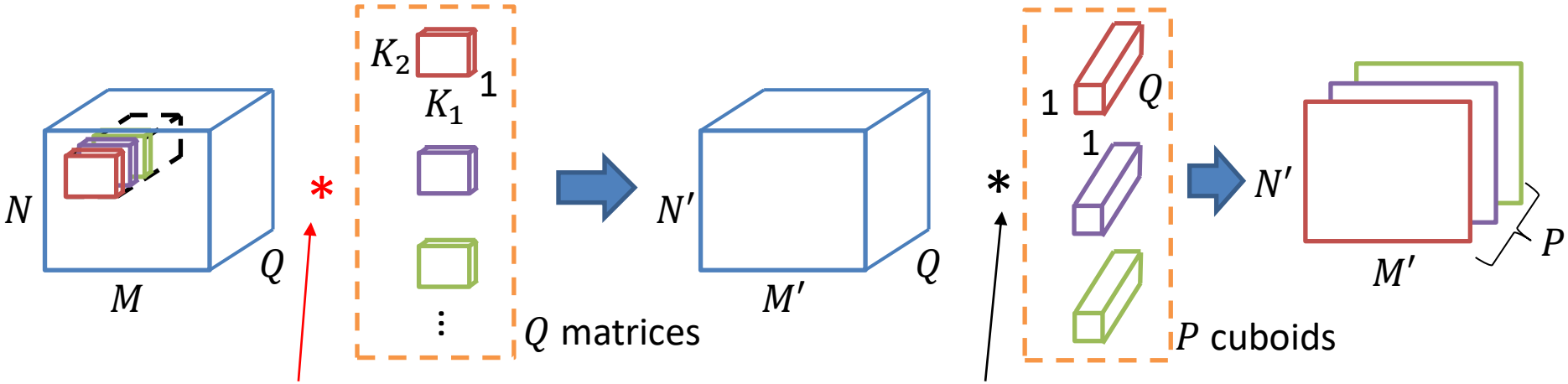
of params: $Q \times K_1 \times K_2$

of params: $Q \times P$

Depthwise separable convolution



cost: $P \times Q \times K_1 \times K_2 \times M' \times N'$



cost: $Q \times K_1 \times K_2 \times M' \times N'$

cost: $P \times Q \times M' \times N'$

Standard conv VS depthwise separable conv

- Number of parameters

$$\frac{\text{depthwise}}{\text{standard}} = \frac{Q \times K_1 \times K_2 + Q \times P}{P \times Q \times K_1 \times K_2} = \frac{1}{P} + \frac{1}{K_1 \times K_2}$$

- Computational cost

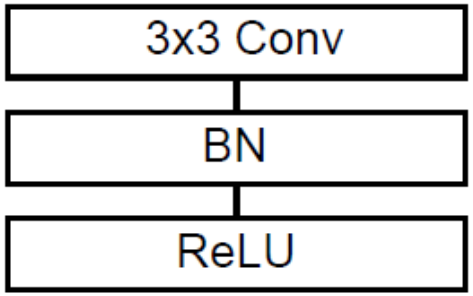
$$\frac{\text{depthwise}}{\text{standard}} = \frac{Q \times K_1 \times K_2 \times M' \times N' + P \times Q \times M' \times N'}{P \times Q \times K_1 \times K_2 \times M' \times N'}$$

$$= \frac{1}{P} + \frac{1}{K_1 \times K_2}$$

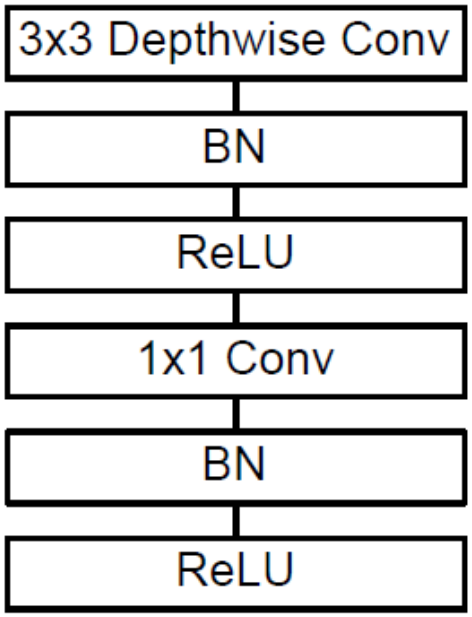
- A typical setting: $P = 100, K_1 = K_2 = 3,$
  - The above ratio is about 1/9

Implementation

A standard conv with BN and ReLU



Corresponding depthwise conv



MobileNet V1

There are extensions of this model: V2 and V3

Howard et al., CVPR 2017

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Results on Imagenet

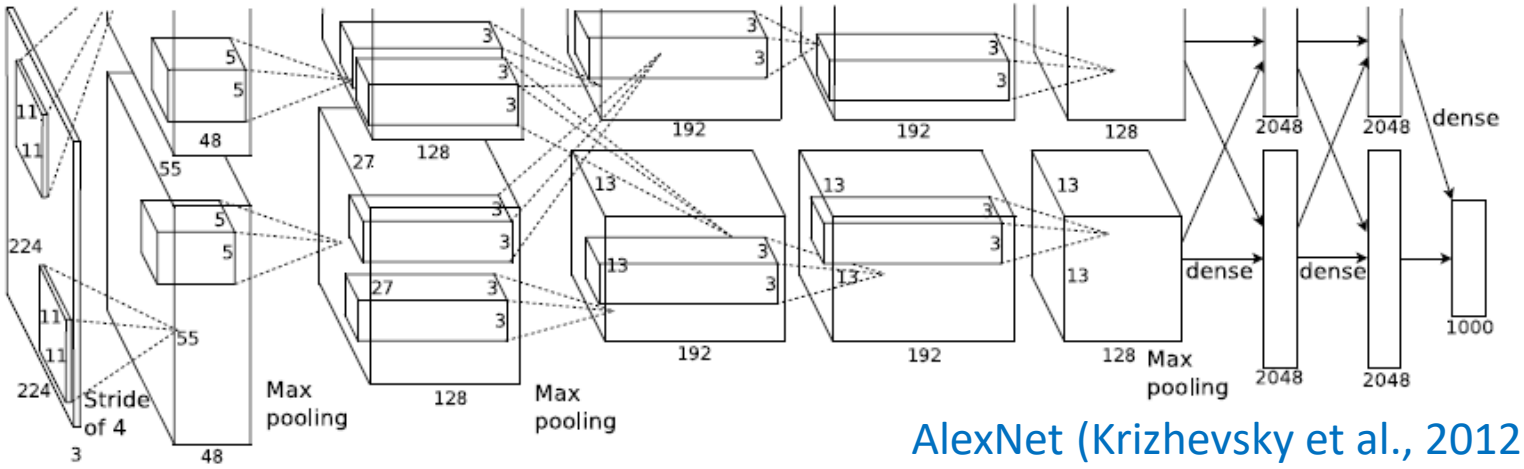
Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

The accuracy is even higher than GoogLeNet

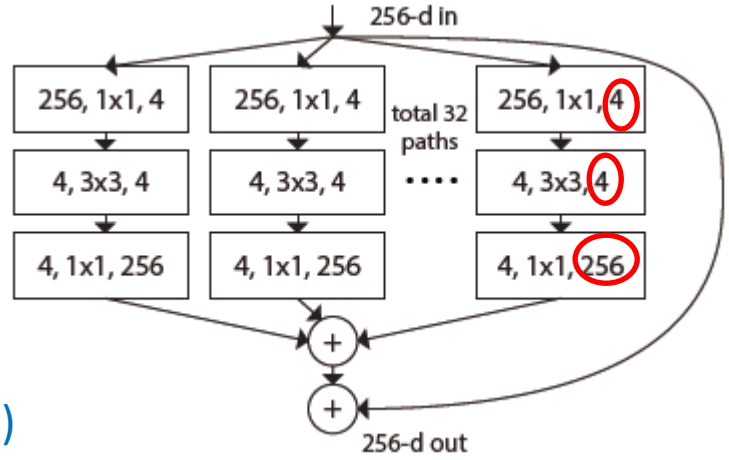
Group convolution

1. Deal with the limited memory of GPUs



AlexNet (Krizhevsky et al., 2012)

2. Reduce the computational cost and enhance the performance

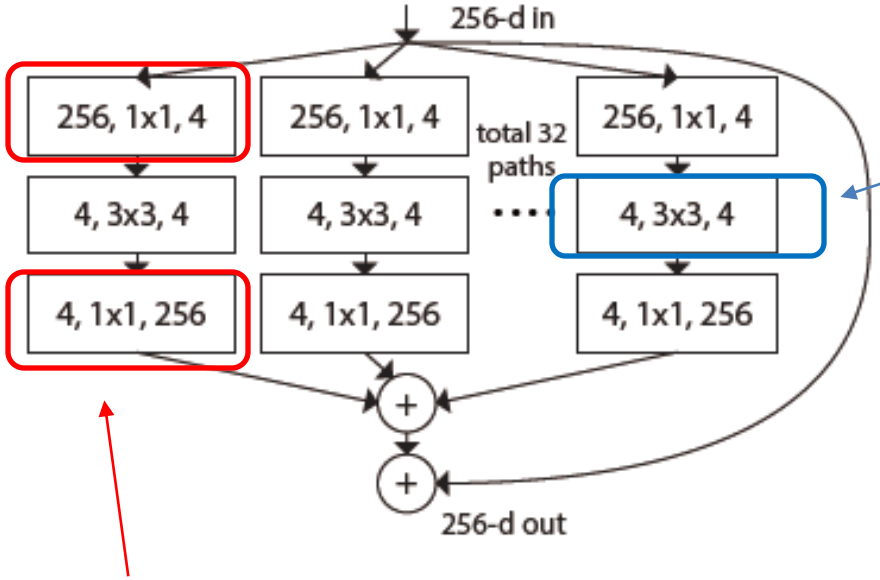


number of channels

Decrease channels then increase

ResNeXt (Xie et al., 2017)

Can we combine depthwise conv and group conv?



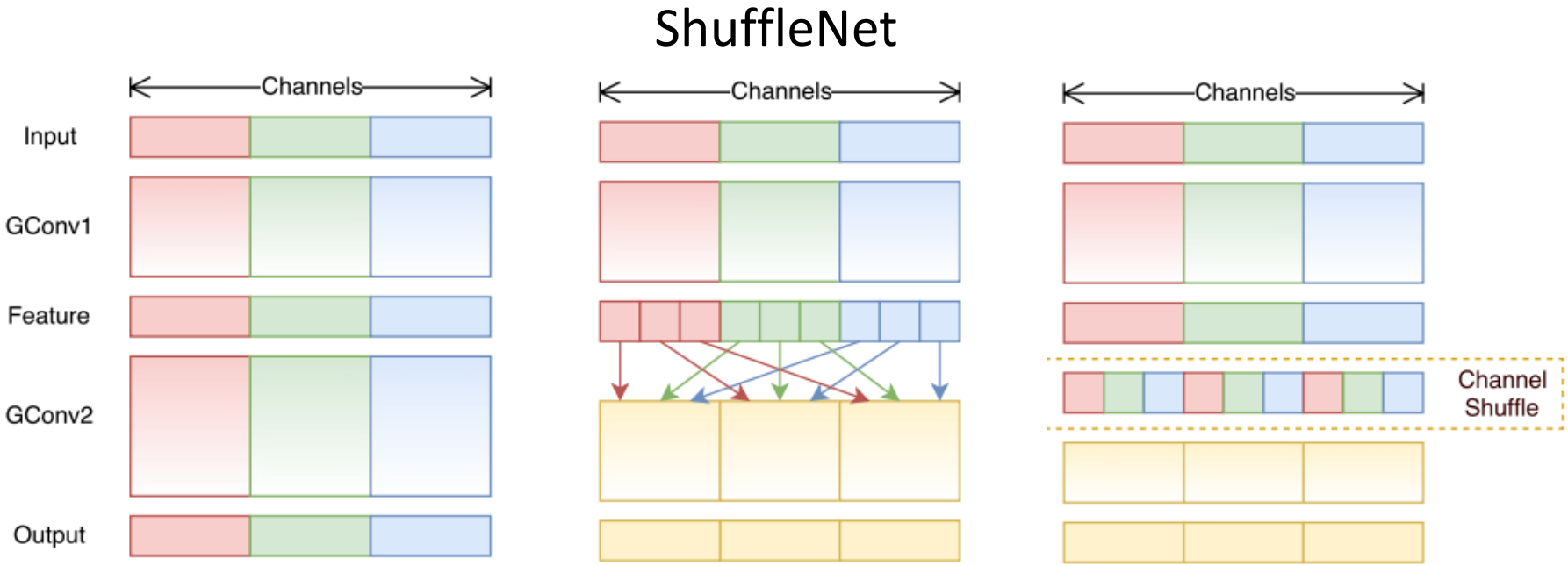
Change all standard 3x3 conv to depthwise conv

1x1 convolutions are not grouped and they dominate the computational cost now!

- For each residual unit in ResNeXt the pointwise convolutions occupy 93.4% multiplication-adds

Group them too!

A new problem and a solution

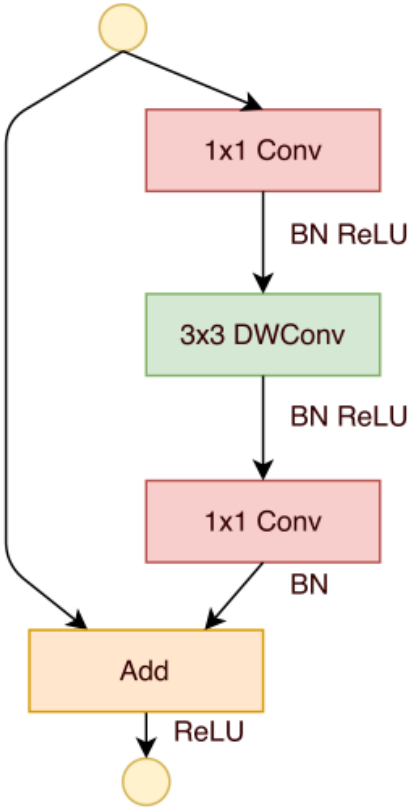


- No crosstalk: it's like a simple combination of many independent subnetworks
- How to make the subnetworks interact with each other?
  - Shuffle the channels

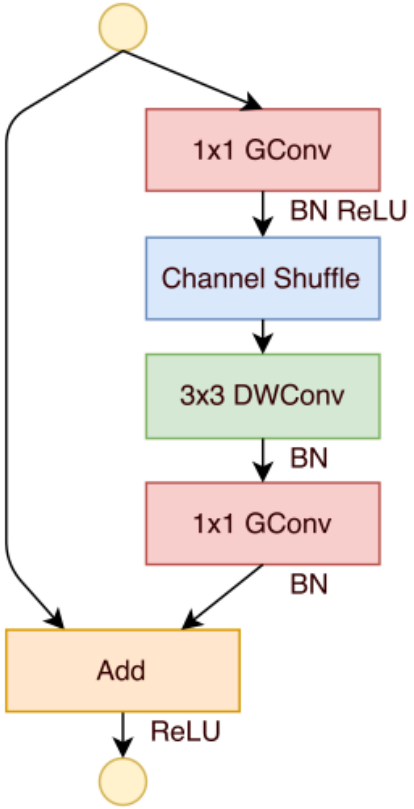
Zhang et al., CVPR 2018

ShuffleNet Units

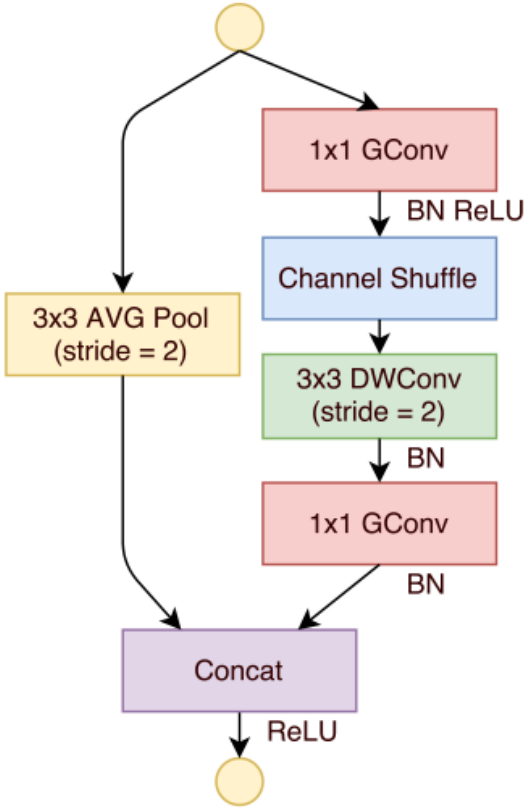
ResNet with Depthwise Conv



ShuffleNet with Group Conv



A variant of ShuffleNet



Zhang et al., CVPR 2018

Results

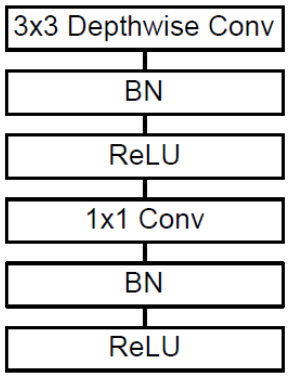
Model	Complexity (MFLOPs)	Cls err. (%)	Δ err. (%)
1.0 MobileNet-224	569	29.4	-
ShuffleNet 2× ($g = 3$)	524	**26.3**	3.1
ShuffleNet 2× (with *SE*[13], $g = 3$)	527	**24.7**	4.7
0.75 MobileNet-224	325	31.6	-
ShuffleNet 1.5× ($g = 3$)	292	**28.5**	3.1
0.5 MobileNet-224	149	36.3	-
ShuffleNet 1× ($g = 8$)	140	**32.4**	3.9
0.25 MobileNet-224	41	49.4	-
ShuffleNet 0.5× ($g = 4$)	38	**41.6**	7.8
ShuffleNet 0.5× (shallow, $g = 3$)	40	42.8	6.6

Summary of part II

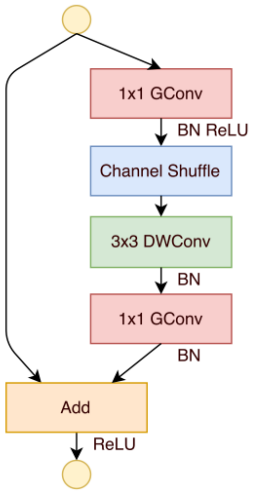
Standard convolution

Depthwise separable convolution

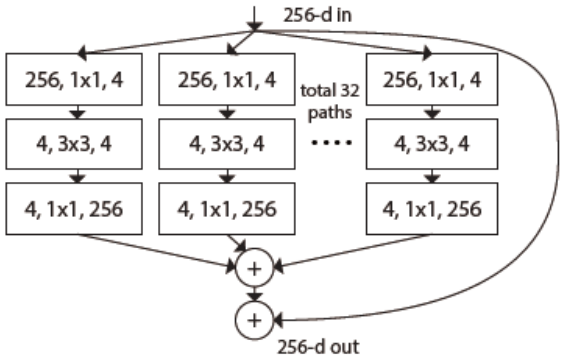
Group convolution



MobileNet



ShuffleNet



ResNeXt

Outline

I. Popular models
II. Lightweight models
III. Model compression and quantization*
IV. Summary

Questions

- Popular neural networks have millions to hundreds of millions parameters
  - Are all parameters important?

Model compression

- The parameters are usually expressed with 32bit
  - Is this necessary?

Model quantization

Network trimming

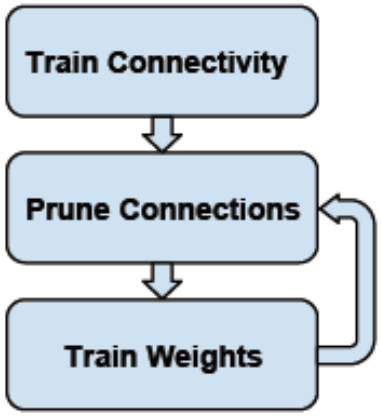


Figure 2: Three-Step Training Pipeline

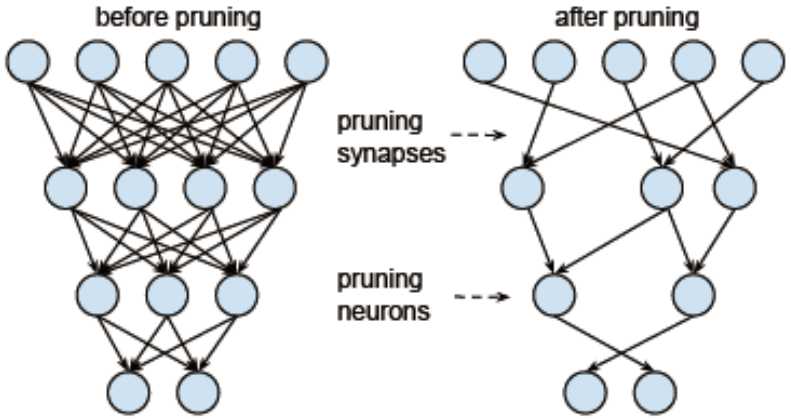


Figure 3: Synapses and neurons before and after pruning

Han et al., NeurIPS 2015

Experiments on MNIST and ImageNet

Network	Top-1 Error	Top-5 Error	Parameters	Compression Rate
LeNet-300-100 Ref	1.64%	-	267K	
LeNet-300-100 Pruned	1.59%	-	**22K**	**12×**
LeNet-5 Ref	0.80%	-	431K	
LeNet-5 Pruned	0.77%	-	**36K**	**12×**
AlexNet Ref	42.78%	19.73%	61M	
AlexNet Pruned	42.77%	19.67%	**6.7M**	**9×**
VGG16 Ref	31.50%	11.32%	138M	
VGG16 Pruned	31.34%	10.88%	**10.3M**	**13×**

On AlexNet

Layer	Weights	FLOP	Act%	Weights%	FLOP%
conv1	35K	211M	88%	84%	84%
conv2	307K	448M	52%	38%	33%
conv3	885K	299M	37%	35%	18%
conv4	663K	224M	40%	37%	14%
conv5	442K	150M	34%	37%	14%
fc1	38M	75M	36%	9%	3%
fc2	17M	34M	40%	9%	3%
fc3	4M	8M	100%	25%	10%
Total	61M	1.5B	54%	**11%**	**30%**

Experiments on MNIST and ImageNet

On VggNet16

Layer	Weights	FLOP	Act%	Weights%	FLOP%
conv1_1	2K	0.2B	53%	58%	58%
conv1_2	37K	3.7B	89%	22%	12%
conv2_1	74K	1.8B	80%	34%	30%
conv2_2	148K	3.7B	81%	36%	29%
conv3_1	295K	1.8B	68%	53%	43%
conv3_2	590K	3.7B	70%	24%	16%
conv3_3	590K	3.7B	64%	42%	29%
conv4_1	1M	1.8B	51%	32%	21%
conv4_2	2M	3.7B	45%	27%	14%
conv4_3	2M	3.7B	34%	34%	15%
conv5_1	2M	925M	32%	35%	12%
conv5_2	2M	925M	29%	29%	9%
conv5_3	2M	925M	19%	36%	11%
fc6	103M	206M	38%	4%	1%
fc7	17M	34M	42%	4%	2%
fc8	4M	8M	100%	23%	9%
total	138M	30.9B	64%	**7.5%**	**21%**

Most parameter savings come from the fully connected layers

Model quantization

Usually, DNN uses single-precision floating point to represent numbers: 32 bits

Can we use fewer bits?

- Many factors to consider:
  - Weights and activations
  - Forward pass and backward pass
  - Fixed-point representation and floating point representation
  - Hardware-specific techniques
    - GPU, CPU, FPGA…

BinaryConnect: consider weights only

Courbariaux et al., NeurIPS 2015

Algorithm 1 SGD training with BinaryConnect. C is the cost function for minibatch and the functions binarize(w) and clip(w) specify how to binarize and clip weights. L is the number of layers.

Require: a minibatch of (inputs, targets), previous parameters w_{t-1} (weights) and b_{t-1} (biases), and learning rate η.

Ensure: updated parameters w_t and b_t.

1. Forward propagation:

$w_b \leftarrow \text{binarize}(w_{t-1})$

For $k = 1$ to L, compute a_k knowing a_{k-1}, w_b and b_{t-1}

2. Backward propagation:

Initialize output layer's activations gradient $\frac{\partial C}{\partial a_L}$

For $k = L$ to 2, compute $\frac{\partial C}{\partial a_{k-1}}$ knowing $\frac{\partial C}{\partial a_k}$ and w_b

3. Parameter update:

Compute $\frac{\partial C}{\partial w_b}$ and $\frac{\partial C}{\partial b_{t-1}}$ knowing $\frac{\partial C}{\partial a_k}$ and a_{k-1}

$w_t \leftarrow \text{clip}(w_{t-1} - \eta \frac{\partial C}{\partial w_b})$

$b_t \leftarrow b_{t-1} - \eta \frac{\partial C}{\partial b_{t-1}}$

Use binary weights only in steps 1&2

Between [-1,1]

A real value version of *w* is needed during training!

Deterministic

$$w_b = \begin{cases} +1 & \text{if } w \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

Stochastic

$$w_b = \begin{cases} +1 & \text{with probability } p = \sigma(w), \\ -1 & \text{with probability } 1 - p. \end{cases}$$

$$\sigma(x) = \text{clip}(\frac{x+1}{2}, 0, 1) = \max(0, \min(1, \frac{x+1}{2}))$$

Experiments

Error rates

Method	MNIST	CIFAR-10	SVHN
No regularizer	$1.30 \pm 0.04\%$	10.64%	2.44%
BinaryConnect (det.)	$1.29 \pm 0.08\%$	9.90%	2.30%
BinaryConnect (stoch.)	$1.18 \pm 0.04\%$	**8.27%**	2.15%
50% Dropout	$1.01 \pm 0.04\%$		
Maxout Networks [29]	0.94%	11.68%	2.47%
Deep L2-SVM [30]	**0.87%**		
Network in Network [31]		10.41%	2.35%
DropConnect [21]			1.94%
Deeply-Supervised Nets [32]		9.78%	**1.92%**

Similar or even better accuracy than standard 32bit versions

BinaryNet: consider weights and activations

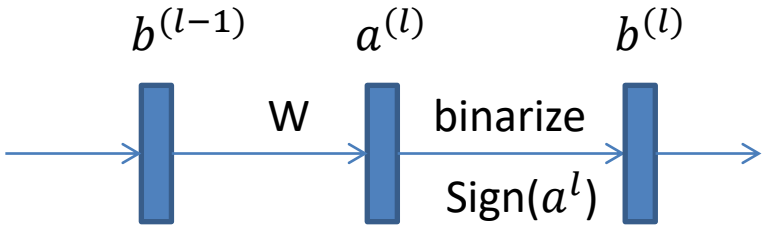
Hubara et al., NeurIPS 2016

- During backpropagation, we need to compute $\partial C/\partial a^{(l)}$

Equivalently $\delta^{(l)}$, as $\delta^{(l)} = \frac{\partial C}{\partial u^{(l)}} = \frac{\partial C}{\partial a^{(l)}}\frac{\partial a^{(l)}}{\partial u^{(l)}} = \frac{\partial C}{\partial a^{(l)}} f'(u^{(l)})$

- With activation binarization

f: Activation fun



$$\frac{\partial C}{\partial a^{(l)}} = \frac{\partial C}{\partial b^{(l)}} \cdot \frac{\partial b^{(l)}}{\partial a^{(l)}}$$

- $\frac{\partial b^{(l)}}{\partial a^{(l)}}$ is zero almost everywhere, making $\frac{\partial C}{\partial a^{(l)}} = 0 = \frac{\partial C}{\partial a^{(l-1)}} = \cdots$

Straight-through estimator

- In the general case, suppose b is binarized using the sign function to a

$$b = \text{sign}(a)$$

- The straight-through estimator of $\partial C/\partial a$ is

$$\frac{\partial C}{\partial a} = \frac{\partial C}{\partial b} \cdot 1_{|a| \leq 1}$$

Note that this preserves the gradient's information and cancels the gradient when $|a|$ is too large

Binarizing the weights and activations

- For binarizing the activations, use the sign function
- For binarizing the weights (same as in the BinaryConnect)
  - Clip them in [-1, 1]
  - Sign transformation
- Other tricks
  - Shift based batch normalization
  - Shift based AdaMax

These techniques can be used in
- both training and testing
- testing only

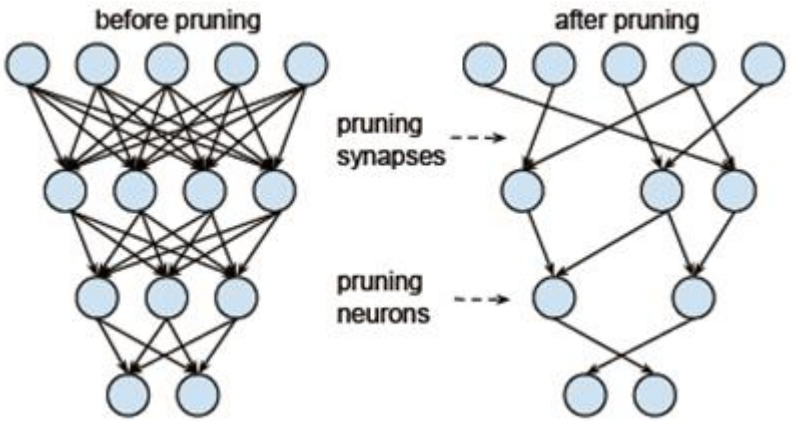
Experiments

Data set	MNIST	SVHN	CIFAR-10
Binarized activations+weights, during training and test			
BNN (Torch7)	1.40%	2.53%	10.15%
BNN (Theano)	0.96%	2.80%	11.40%
Committee Machines' Array (Baldassi et al., 2015)	1.35%	-	-
Binarized weights, during training and test			
BinaryConnect (Courbariaux et al., 2015)	1.29± 0.08%	2.30%	9.90%
Binarized activations+weights, during test			
EBP (Cheng et al., 2015)	2.2± 0.1%	-	-
Bitwise DNNs (Kim & Smaragdis, 2016)	1.33%	-	-
Ternary weights, binary activations, during test			
(Hwang & Sung, 2014)	1.45%	-	-
No binarization (standard results)			
Maxout Networks (Goodfellow et al.)	0.94%	2.47%	11.68%
Network in Network (Lin et al.)	-	2.35%	10.41%
Gated pooling (Lee et al., 2015)	-	1.69%	7.62%

Summary of part III

Model compression

Model quantization



Network trimming

- BinaryConnect
- BinaryNet

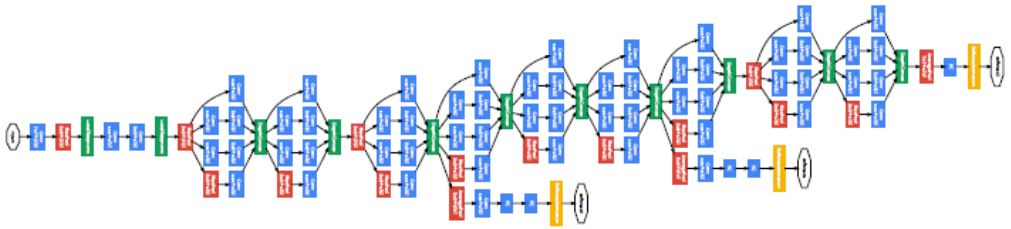
Outline

I. Popular models
II. Lightweight models
III. Model compression and quantization*
IV. Summary

Summary of this lecture

Knowledge

I. Popular models



AlexNet

ResNet

GoogLeNet

VGGNet

DenseNet

II. Lightweight models

- Depthwise separable convolution
- Group convolution

MobileNet

ShuffleNet

ResNeXt

III. Compression and quantization*

- Model compression
- Model quantization

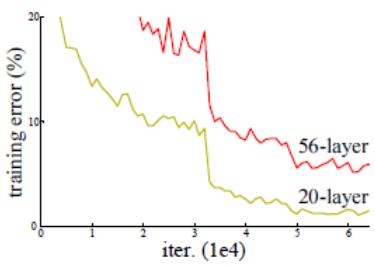
Network trimming

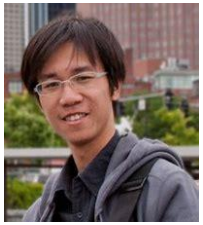
BinaryConnect

BinaryNet

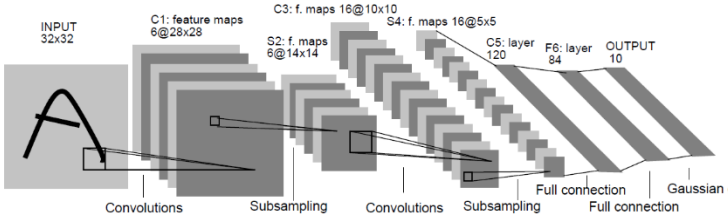
Summary of this lecture

Capability and value





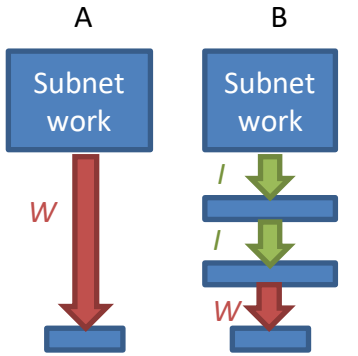
Kaiming He



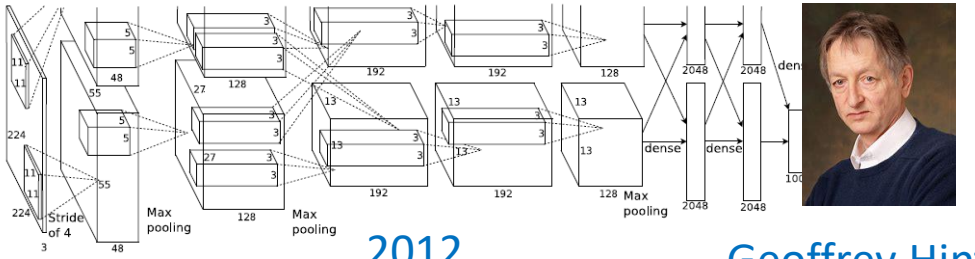
1989-1998



Yann LeCun



Best CVPR 2016 paper



2012

Geoffrey Hinton

Mark the resurgence of deep learning

- 善于分析：从失败中找问题
- 勤于动手：不要不屑于做看起来没有创新的事情

Prepare for the next lecture

- Form groups of 2-3 and every group prepares a 5-minute presentation with slides for one of the following papers
  - Hu, Shen, Sun, “Squeeze-and-Excitation Networks, ” CVPR 2018
  - Li, Wang, Hu, Yang, “Selective Kernel Networks,” CVPR 2019

Homework 3

- MLP&CNN for MNIST and CIFAR-10
  - You can use either Pytorch or other software
  - Deadline: Nov 10 (Monday noon) 12:00